

講義 11：多表 JOIN 與 CORRESPONDING FIELDS

對應練習：[ex11](#) | 答案程式：[ZR_TR11_JOIN](#)

本講重點

- 為什麼要 JOIN：報表幾乎都要跨表撈欄位
- **INNER JOIN** 與 **LEFT OUTER JOIN** 的差別
- 語法細節：別名 **AS**、欄位前綴 **~**、**ON** 條件
- **INTO CORRESPONDING FIELDS OF TABLE**：依欄位名對應
- **FOR ALL ENTRIES** 簡介與著名的空表陷阱
- 效能觀念：JOIN vs 迴圈中 SELECT

1. 為什麼要 JOIN

需求：「列出每個航班，含航空公司名稱」。航班在 SFLIGHT，公司名稱在 SCARR——單表 SELECT 拿不齊。
最糟的寫法是迴圈裡逐筆查：

```
* 反面教材：N 筆航班 = N 次資料庫往返，大表直接跑不動
LOOP AT gt_flights INTO gs_flight.
  SELECT SINGLE carrname FROM scarr INTO gv_name
    WHERE carrid = gs_flight-carrid.
ENDLOOP.
```

JOIN 讓資料庫一次把兩張表串好回傳——這是本講存在的理由。

2. INNER JOIN

```
TYPES: BEGIN OF ty_flight,
    carrid   TYPE sflight-carrid,
    connid   TYPE sflight-connid,
    fldate   TYPE sflight-fldate,
    carrname TYPE scarr-carrname,      " 來自另一張表的欄位
END OF ty_flight.
DATA gt_flights TYPE STANDARD TABLE OF ty_flight.

SELECT f~carrid f~connid f~fldate c~carrname
  INTO CORRESPONDING FIELDS OF TABLE gt_flights
  FROM sflight AS f
  INNER JOIN scarr AS c ON f~carrid = c~carrid
  WHERE f~fldate >= '20260101'.
```

語法要點：

- **AS f** / **AS c**：表別名。之後每個欄位都要標明來源：**f~carrid**、**c~carrname**（波浪號 ~，不是 -）。

- **ON** 是串表條件（兩表怎麼對上），**WHERE** 是過濾條件——語意不同，別混在一起。ON 可以多條件：**ON f~carrid = c~carrid AND f~connid = c~connid**。
- **INNER JOIN** 的語意：兩邊都對得上才輸出。SFLIGHT 有但 SCARR 沒有的公司代碼，那些航班整筆消失。

3. LEFT OUTER JOIN

「左表全留，右表對不上就給初始值」：

```
SELECT c~carrid c~carrname f~connid f~fldate
  INTO CORRESPONDING FIELDS OF TABLE gt_result
FROM scarr AS c
LEFT OUTER JOIN sflight AS f ON c~carrid = f~carrid.
```

結果：每家航空公司都在（左表 SCARR 全留）；沒有任何航班的公司，**connid/fldate** 是初始值（空白/00000000）。

選擇原則：「沒對到的要不要出現在報表上？」要 → **LEFT OUTER**；不要 → **INNER**。例如「各公司營收統計，沒航班的公司也要列出 0」就是 **LEFT OUTER** 的場景。

限制：**LEFT OUTER JOIN** 的 **WHERE** 對右表欄位下條件會把 **NULL** 列過濾掉，效果變回 **INNER**——右表條件盡量放在 **ON**。

4. INTO 的兩種對應方式

寫法	對應規則	風險
INTO TABLE gt	依順序：第 1 個欄位塞結構第 1 欄...	順序錯 = 資料錯位（不報錯）
INTO CORRESPONDING FIELDS OF TABLE gt	依欄位名：同名才塞	欄名拼錯 = 該欄默默留空

JOIN 的結果結構是自訂的、欄位東拼西湊，建議一律用 **CORRESPONDING FIELDS**：順序自由、可讀性高；代價是欄位名必須跟 **SELECT** 清單一致（別名欄位可用 **AS**：**SELECT f~price AS ticket_price ...**）。

5. FOR ALL ENTRIES（先認識，維護會遇到）

JOIN 之外的另一種跨表手法：先撈第一張表進內表，再用內表內容當第二張表的條件：

```
IF gt_flights IS NOT INITIAL.      " 這個 IF 是保命符！
  SELECT carrid carrname FROM scarr
    INTO TABLE gt_carriers
  FOR ALL ENTRIES IN gt_flights
    WHERE carrid = gt_flights-carrid.
ENDIF.
```

兩個必知：

- 空表陷阱：`gt_flights` 是空的時，FOR ALL ENTRIES 的 WHERE 整個失效 = 全表撈回。前面那個 `IF ... IS NOT INITIAL` 不是可選的。
- 結果會自動去除完全重複的列。

新程式優先 JOIN；FOR ALL ENTRIES 用在 JOIN 不方便的場景（如來源是加工過的內表），舊程式裡極常見。

6. 效能觀念小結

寫法	資料庫往返	評價
LOOP 內 SELECT SINGLE	N 次	禁止（測試資料看不出慢，上線就爆）
SELECT + FOR ALL ENTRIES	2 次	可用，記得空表防呆
INNER / LEFT OUTER JOIN	1 次	首選

7. 常見錯誤與陷阱

症狀	原因
欄位前綴報語法錯誤	JOIN 裡欄位要用 <code>f~carrid (~)</code> ，不是 <code>-</code>
某些資料整筆不見	INNER JOIN 對不上就丟——確認是否該用 LEFT OUTER
CORRESPONDING 後某欄全空	結構欄位名跟 SELECT 欄位名（或 AS 別名）不一致
LEFT OUTER 加了右表 WHERE 後行為像 INNER	右表條件要放 ON，不放 WHERE
FOR ALL ENTRIES 慢到懷疑人生	驅動內表是空的，全表撈回——補 IS NOT INITIAL 檢查

8. 課堂練習

完成 [ex11](#)：INNER JOIN 撈「航班 + 公司名稱」、LEFT OUTER JOIN 觀察沒航班的公司、比較兩種 INTO 的行為差異。